

LeetCode Module 1

Complete Solution Guide

60 problems · theorem/technique · explanation · pseudocode · full Java code

Every problem includes the core theorem or technique it relies on, a plain-language explanation of the approach, language-agnostic pseudocode, and a complete, ready-to-run Java solution.

Contents

Level 1 (Absolute Beginner)

1. Fizz Buzz (#412)
2. Palindrome Number (#9)
3. Number of Steps to Reduce a Number to Zero (#1342)
4. Richest Customer Wealth (#1672)
5. Running Sum of 1D Array (#1480)
6. Find Numbers with Even Number of Digits (#1295)
7. Shuffle the Array (#1470)
8. Concatenation of Array (#1929)
9. Build Array from Permutation (#1920)
10. Kids With Greatest Number of Candies (#1431)

Level 2 (Arrays)

11. Two Sum (#1)
12. Best Time to Buy and Sell Stock (#121)
13. Remove Duplicates from Sorted Array (#26)
14. Remove Element (#27)
15. Maximum Subarray (#53)
16. Plus One (#66)
17. Merge Sorted Array (#88)
18. Contains Duplicate (#217)
19. Majority Element (#169)
20. Find Pivot Index (#724)

Level 3 (Strings)

21. Valid Anagram (#242)
22. Reverse String (#344)
23. Reverse Words in a String III (#557)
24. Length of Last Word (#58)
25. Longest Common Prefix (#14)
26. Roman to Integer (#13)
27. Valid Palindrome (#125)
28. First Unique Character in a String (#387)
29. Reverse Vowels of a String (#345)
30. Add Binary (#67)

Level 4 (Searching)

31. Binary Search (#704)
32. Search Insert Position (#35)
33. Guess Number Higher or Lower (#374)

- 34. Square Root(x) (#69)
- 35. First Bad Version (#278)

Level 5 (Recursion)

- 36. Fibonacci Number (#509)
- 37. Power of Two (#231)
- 38. Climbing Stairs (#70)
- 39. Pow(x,n) (#50)
- 40. Recursive Binary Search

Level 6 (Intermediate)

- 41. Pascal's Triangle (#118)
- 42. Pascal's Triangle II (#119)
- 43. Happy Number (#202)
- 44. Missing Number (#268)
- 45. Move Zeroes (#283)
- 46. Rotate Array (#189)
- 47. Third Maximum Number (#414)
- 48. Find All Numbers Disappeared in Array (#448)
- 49. Valid Mountain Array (#941)
- 50. Monotonic Array (#896)

Hardest Questions Possible with Only Module 1 Knowledge

- 51. Longest Palindromic Substring (#5)
- 52. Zigzag Conversion (#6)
- 53. String to Integer (atoi) (#8)
- 54. Container With Most Water (#11)
- 55. 3Sum (#15)
- 56. Generate Parentheses (#22)
- 57. Letter Combinations of a Phone Number (#17)
- 58. Permutations (#46)
- 59. Combination Sum (#39)
- 60. Sudoku Solver (#37)

Level 1 (Absolute Beginner)

1. Fizz Buzz

LeetCode #412 | Difficulty: **Easy**

Theorem / Technique Used

Modulo Arithmetic (Divisibility Check)

Explanation

Loop from 1 to n. For each number, check divisibility by both 3 and 5 first (since 15 covers both), then 3 alone, then 5 alone, otherwise print the number itself. Order of checks matters: checking the combined condition (divisible by 15) before the individual ones avoids only printing 'Fizz' or 'Buzz' by mistake.

Pseudocode

```
for i = 1 to n:
    if i % 15 == 0:
        print("FizzBuzz")
    else if i % 3 == 0:
        print("Fizz")
    else if i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

Full Code (Java)

```
class Solution {
    public List<String> fizzBuzz(int n) {
        List<String> result = new ArrayList<>();
        for (int i = 1; i <= n; i++) {
            if (i % 15 == 0) result.add("FizzBuzz");
            else if (i % 3 == 0) result.add("Fizz");
            else if (i % 5 == 0) result.add("Buzz");
            else result.add(String.valueOf(i));
        }
        return result;
    }
}
```

2. Palindrome Number

LeetCode #9 | Difficulty: **Easy**

Theorem / Technique Used

Digit Reversal via Modulo/Division

Explanation

A number is a palindrome if it reads the same forwards and backwards. Handle the edge case of negative numbers (never palindromes because of the '-' sign) directly. Otherwise, reverse the digits of the number mathematically using modulo (%) and integer division (/), then compare the reversed number with the original.

Pseudocode

```
function isPalindrome(x):
    if x < 0:
        return false
    original = x
    reversed = 0
    while x > 0:
        digit = x % 10
        reversed = reversed * 10 + digit
        x = x / 10    // integer division
    return reversed == original
```

Full Code (Java)

```
class Solution {
    public boolean isPalindrome(int x) {
        if (x < 0) return false;
        long original = x;
        long reversed = 0;
        while (x > 0) {
            int digit = x % 10;
            reversed = reversed * 10 + digit;
            x /= 10;
        }
        return reversed == original;
    }
}
```

3. Number of Steps to Reduce a Number to Zero

LeetCode #1342 | Difficulty: **Easy**

Theorem / Technique Used

Greedy Simulation

Explanation

Simulate the process directly. Keep a counter of steps. While the number is greater than zero, check if it is even or odd: divide even numbers by 2, subtract 1 from odd numbers. Increment the step counter each time and stop once the number reaches zero.

Pseudocode

```
function numberOfSteps(num):
    steps = 0
    while num > 0:
        if num % 2 == 0:
            num = num / 2
        else:
            num = num - 1
            steps = steps + 1
    return steps
```

Full Code (Java)

```
class Solution {
    public int numberOfSteps(int num) {
        int steps = 0;
        while (num > 0) {
            if (num % 2 == 0) num /= 2;
            else num -= 1;
            steps++;
        }
        return steps;
    }
}
```

4. Richest Customer Wealth

LeetCode #1672 | Difficulty: **Easy**

Theorem / Technique Used

Row-wise Reduction (Linear Scan)

Explanation

Each row of the 2D array represents one customer's accounts. For every row, sum all the values in that row to get that customer's total wealth. Track the maximum total sum seen across all rows and return it at the end.

Pseudocode

```
function maximumWealth(accounts):
    maxWealth = 0
    for each row in accounts:
        rowSum = 0
        for each value in row:
            rowSum = rowSum + value
        maxWealth = max(maxWealth, rowSum)
    return maxWealth
```

Full Code (Java)

```
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;
        for (int[] row : accounts) {
            int rowSum = 0;
            for (int value : row) rowSum += value;
            maxWealth = Math.max(maxWealth, rowSum);
        }
        return maxWealth;
    }
}
```

5. Running Sum of 1D Array

LeetCode #1480 | Difficulty: **Easy**

Theorem / Technique Used

Prefix Sum

Explanation

This is a classic prefix-sum problem: each output element is the sum of all input elements up to and including that index. Instead of recomputing sums from scratch, build the result incrementally: $\text{runningSum}[i] = \text{runningSum}[i-1] + \text{nums}[i]$. This can even be done in place, overwriting the input array.

Pseudocode

```
function runningSum(nums):
    for i = 1 to length(nums) - 1:
        nums[i] = nums[i] + nums[i-1]
    return nums
```

Full Code (Java)

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
        return nums;
    }
}
```

6. Find Numbers with Even Number of Digits

LeetCode #1295 | Difficulty: **Easy**

Theorem / Technique Used

Digit Counting

Explanation

For each number in the array, determine how many digits it has (either by converting to a string and measuring its length, or by repeatedly dividing by 10 and counting). Count how many numbers have an even digit count.

Pseudocode

```
function findNumbers(nums):
    count = 0
    for each num in nums:
        digits = 0
        n = num
        if n == 0:
            digits = 1
        while n > 0:
            n = n / 10
            digits = digits + 1
        if digits % 2 == 0:
            count = count + 1
    return count
```

Full Code (Java)

```
class Solution {
    public int findNumbers(int[] nums) {
        int count = 0;
        for (int num : nums) {
            int digits = (num == 0) ? 1 : 0;
            int n = num;
            while (n > 0) {
                n /= 10;
                digits++;
            }
            if (digits % 2 == 0) count++;
        }
        return count;
    }
}
```

7. Shuffle the Array

LeetCode #1470 | Difficulty: **Easy**

Theorem / Technique Used

Two Pointer (Index Offset)

Explanation

The array is given as $[x_1, \dots, x_n, y_1, \dots, y_n]$ and must be rearranged into $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$. Use two pointers: one starting at index 0 for the x-values and one starting at index n for the y-values. Build the result array by alternately picking one value from each pointer and advancing both.

Pseudocode

```
function shuffle(nums, n):
    result = new array of size 2*n
    for i = 0 to n - 1:
        result[2*i] = nums[i]
        result[2*i + 1] = nums[n + i]
    return result
```

Full Code (Java)

```
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] result = new int[2 * n];
        for (int i = 0; i < n; i++) {
            result[2 * i] = nums[i];
            result[2 * i + 1] = nums[n + i];
        }
        return result;
    }
}
```

8. Concatenation of Array

LeetCode #1929 | Difficulty: **Easy**

Theorem / Technique Used

Array Duplication

Explanation

The task is to build a new array that is simply the original array written out twice, back to back. Loop through the array once and copy each element to two positions in the result: index i and index i+n.

Pseudocode

```
function getConcatenation(nums):
    n = length(nums)
    result = new array of size 2*n
    for i = 0 to n - 1:
        result[i] = nums[i]
        result[i + n] = nums[i]
    return result
```

Full Code (Java)

```
class Solution {
    public int[] getConcatenation(int[] nums) {
        int n = nums.length;
        int[] result = new int[2 * n];
        for (int i = 0; i < n; i++) {
            result[i] = nums[i];
            result[i + n] = nums[i];
        }
        return result;
    }
}
```

9. Build Array from Permutation

LeetCode #1920 | Difficulty: **Easy**

Theorem / Technique Used

Index Mapping / Permutation Composition

Explanation

For each index i , the answer must contain $\text{nums}[\text{nums}[i]]$. Build a new result array so the original input isn't overwritten mid-computation (overwriting in place would corrupt values still needed by later lookups unless done carefully with encoding tricks).

Pseudocode

```
function buildArray(nums):
    n = length(nums)
    result = new array of size n
    for i = 0 to n - 1:
        result[i] = nums[nums[i]]
    return result
```

Full Code (Java)

```
class Solution {
    public int[] buildArray(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        for (int i = 0; i < n; i++) {
            result[i] = nums[nums[i]];
        }
        return result;
    }
}
```

10. Kids With Greatest Number of Candies

LeetCode #1431 | Difficulty: **Easy**

Theorem / Technique Used

Max Tracking + Linear Scan

Explanation

First, find the maximum number of candies among all kids. Then, for every kid, check if their current candies plus the extra candies would be at least equal to that maximum. If so, mark true; otherwise mark false.

Pseudocode

```
function kidsWithCandies(candies, extraCandies):
    maxCandies = max(candies)
    result = new boolean array
    for each c in candies:
        result.append(c + extraCandies >= maxCandies)
    return result
```

Full Code (Java)

```
class Solution {
    public List<Boolean> kidsWithCandies(int[] candies, int extraCandies) {
        int maxCandies = 0;
        for (int c : candies) maxCandies = Math.max(maxCandies, c);
        List<Boolean> result = new ArrayList<>();
        for (int c : candies) {
            result.add(c + extraCandies >= maxCandies);
        }
        return result;
    }
}
```

Level 2 (Arrays)

11. Two Sum

LeetCode #1 | Difficulty: **Easy**

Theorem / Technique Used

Brute Force (Nested Loop)

Explanation

Module 1 approach: brute force. Use two nested loops — the outer loop fixes the first number, the inner loop tries every other number after it, checking if the pair sums to the target. This is $O(n^2)$ but only requires basic loop and comparison knowledge (a hash-map approach is faster but belongs to a later module).

Pseudocode

```
function twoSum(nums, target):
    for i = 0 to length(nums) - 1:
        for j = i + 1 to length(nums) - 1:
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

Full Code (Java)

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{};
    }
}
```

12. Best Time to Buy and Sell Stock

LeetCode #121 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Single-Pass Min Tracking (Greedy)

Explanation

Track the minimum price seen so far while scanning through the prices left to right. At each day, compute the profit if you sold on that day (current price minus the minimum price so far) and keep the best (maximum) profit found. This avoids checking every buy/sell pair, giving a single $O(n)$ pass.

Pseudocode

```
function maxProfit(prices):
    minPrice = infinity
    maxProfit = 0
    for each price in prices:
        if price < minPrice:
            minPrice = price
        else if price - minPrice > maxProfit:
            maxProfit = price - minPrice
    return maxProfit
```

Full Code (Java)

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
        for (int price : prices) {
            if (price < minPrice) {
                minPrice = price;
            } else if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }
        return maxProfit;
    }
}
```

13. Remove Duplicates from Sorted Array

LeetCode #26 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two Pointer (Slow/Fast)

Explanation

Because the array is sorted, all duplicates of a value sit next to each other. Use a slow pointer (position to write the next unique value) and a fast pointer (scans through the array). Whenever the fast pointer finds a value different from the last written value, write it at the slow pointer's position and advance the slow pointer.

Pseudocode

```
function removeDuplicates(nums):
    if length(nums) == 0:
        return 0
    slow = 0
    for fast = 1 to length(nums) - 1:
        if nums[fast] != nums[slow]:
            slow = slow + 1
            nums[slow] = nums[fast]
    return slow + 1 // new length
```

Full Code (Java)

```
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;
        int slow = 0;
        for (int fast = 1; fast < nums.length; fast++) {
            if (nums[fast] != nums[slow]) {
                slow++;
                nums[slow] = nums[fast];
            }
        }
        return slow + 1;
    }
}
```

14. Remove Element

LeetCode #27 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two Pointer (Slow/Fast)

Explanation

Use a slow pointer to track where the next kept (non-matching) element should be written, and a fast pointer to scan every element. Whenever the fast pointer lands on a value that is NOT equal to val, copy it to the slow pointer's index and advance the slow pointer.

Pseudocode

```
function removeElement(nums, val):
    slow = 0
    for fast = 0 to length(nums) - 1:
        if nums[fast] != val:
            nums[slow] = nums[fast]
            slow = slow + 1
    return slow // new length
```

Full Code (Java)

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int slow = 0;
        for (int fast = 0; fast < nums.length; fast++) {
            if (nums[fast] != val) {
                nums[slow] = nums[fast];
                slow++;
            }
        }
        return slow;
    }
}
```

15. Maximum Subarray

LeetCode #53 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Kadane's Algorithm (Dynamic Programming)

Explanation

Kadane's Algorithm: walk through the array while keeping a running sum of the 'current' subarray. If the running sum ever becomes negative, it can only hurt any future subarray, so reset it to the current element instead of carrying negative value forward. Track the maximum running sum seen at any point as the answer.

Pseudocode

```
function maxSubArray(nums):
    currentSum = nums[0]
    maxSum = nums[0]
    for i = 1 to length(nums) - 1:
        currentSum = max(nums[i], currentSum + nums[i])
        maxSum = max(maxSum, currentSum)
    return maxSum
```

Full Code (Java)

```
class Solution {
    public int maxSubArray(int[] nums) {
        int currentSum = nums[0];
        int maxSum = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
        return maxSum;
    }
}
```

16. Plus One

LeetCode #66 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Carry Propagation (Simulated Arithmetic)

Explanation

The digits array represents a large number, most significant digit first. Start from the last digit and add 1. If a digit becomes 10, set it to 0 and carry 1 into the digit to its left (this is standard elementary-school addition with carrying). If the carry propagates past the first digit, a new leading 1 must be inserted at the front.

Pseudocode

```
function plusOne(digits):
    for i = length(digits) - 1 down to 0:
        if digits[i] < 9:
            digits[i] = digits[i] + 1
            return digits
        digits[i] = 0
    // if we fall through, every digit was 9
    newDigits = new array of size length(digits)+1, filled with 0
    newDigits[0] = 1
    return newDigits
```

Full Code (Java)

```
class Solution {
    public int[] plusOne(int[] digits) {
        for (int i = digits.length - 1; i >= 0; i--) {
            if (digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }
        int[] newDigits = new int[digits.length + 1];
        newDigits[0] = 1;
        return newDigits;
    }
}
```

17. Merge Sorted Array

LeetCode #88 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Three Pointer (Merge from the Back)

Explanation

nums1 has extra empty space at the end to hold the merged result. Merge from the back rather than the front, using three pointers: one at the last real element of nums1, one at the last element of nums2, and one at the very last slot of nums1. Repeatedly place the larger of the two candidates into the last open slot and move pointers left, avoiding overwriting values in nums1 that haven't been compared yet.

Pseudocode

```
function merge(nums1, m, nums2, n):
    i = m - 1          // last real element in nums1
    j = n - 1          // last element in nums2
    k = m + n - 1      // last slot of nums1
    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[k] = nums1[i]
            i = i - 1
        else:
            nums1[k] = nums2[j]
            j = j - 1
        k = k - 1
```

Full Code (Java)

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1;
        int j = n - 1;
        int k = m + n - 1;
        while (j >= 0) {
            if (i >= 0 && nums1[i] > nums2[j]) {
                nums1[k--] = nums1[i--];
            } else {
                nums1[k--] = nums2[j--];
            }
        }
    }
}
```

18. Contains Duplicate

LeetCode #217 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Brute Force (Nested Loop)

Explanation

Module 1 (brute force) approach: compare every pair of elements using two nested loops. If any two distinct positions hold the same value, return true. If the loops finish without finding a match, return false. (A hash-set gives $O(n)$ but uses a later-module data structure.)

Pseudocode

```
function containsDuplicate(nums):  
    for i = 0 to length(nums) - 1:  
        for j = i + 1 to length(nums) - 1:  
            if nums[i] == nums[j]:  
                return true  
    return false
```

Full Code (Java)

```
class Solution {  
    public boolean containsDuplicate(int[] nums) {  
        for (int i = 0; i < nums.length; i++) {  
            for (int j = i + 1; j < nums.length; j++) {  
                if (nums[i] == nums[j]) return true;  
            }  
        }  
        return false;  
    }  
}
```

19. Majority Element

LeetCode #169 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Boyer-Moore Voting Algorithm

Explanation

Boyer-Moore Voting Algorithm: maintain a 'candidate' and a 'count'. Walk through the array; if count is 0, pick the current element as the new candidate. If the current element equals the candidate, increment count, otherwise decrement it. Because the majority element appears more than $n/2$ times, it always survives as the final candidate.

Pseudocode

```
function majorityElement(nums):
    count = 0
    candidate = null
    for each num in nums:
        if count == 0:
            candidate = num
        if num == candidate:
            count = count + 1
        else:
            count = count - 1
    return candidate
```

Full Code (Java)

```
class Solution {
    public int majorityElement(int[] nums) {
        int count = 0;
        Integer candidate = null;
        for (int num : nums) {
            if (count == 0) candidate = num;
            count += (num == candidate) ? 1 : -1;
        }
        return candidate;
    }
}
```

20. Find Pivot Index

LeetCode #724 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Prefix Sum / Total-Minus-Left Trick

Explanation

Compute the total sum of the array first. Then walk through the array while keeping a running left-sum. At each index, the right-sum equals $\text{totalSum} - \text{leftSum} - \text{nums}[i]$. If leftSum equals this right-sum, that index is the pivot. Add the current value to leftSum after checking, so it is ready for the next iteration.

Pseudocode

```
function pivotIndex(nums):
    totalSum = sum(nums)
    leftSum = 0
    for i = 0 to length(nums) - 1:
        rightSum = totalSum - leftSum - nums[i]
        if leftSum == rightSum:
            return i
        leftSum = leftSum + nums[i]
    return -1
```

Full Code (Java)

```
class Solution {
    public int pivotIndex(int[] nums) {
        int totalSum = 0;
        for (int n : nums) totalSum += n;
        int leftSum = 0;
        for (int i = 0; i < nums.length; i++) {
            int rightSum = totalSum - leftSum - nums[i];
            if (leftSum == rightSum) return i;
            leftSum += nums[i];
        }
        return -1;
    }
}
```

Level 3 (Strings)

21. Valid Anagram

LeetCode #242 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Frequency Counting (Counting Sort Style)

Explanation

Two strings are anagrams if they contain exactly the same characters with the same frequencies. Count the frequency of each character in the first string (incrementing a counter array), then decrement the same counters while scanning the second string. If every counter ends at zero and the lengths matched, the strings are anagrams.

Pseudocode

```
function isAnagram(s, t):
    if length(s) != length(t):
        return false
    count = array of 26 zeros
    for each ch in s:
        count[ch - 'a'] = count[ch - 'a'] + 1
    for each ch in t:
        count[ch - 'a'] = count[ch - 'a'] - 1
    for each c in count:
        if c != 0:
            return false
    return true
```

Full Code (Java)

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++;
        for (char ch : t.toCharArray()) count[ch - 'a']--;
        for (int c : count) {
            if (c != 0) return false;
        }
        return true;
    }
}
```

22. Reverse String

LeetCode #344 | Difficulty: **Easy**

Theorem / Technique Used

Two Pointer

Explanation

Use the two-pointer technique: one pointer at the start, one at the end of the character array. Swap the characters at both pointers, then move the left pointer forward and the right pointer backward. Repeat until the pointers meet or cross.

Pseudocode

```
function reverseString(s):
    left = 0
    right = length(s) - 1
    while left < right:
        swap(s[left], s[right])
        left = left + 1
        right = right - 1
```

Full Code (Java)

```
class Solution {
    public void reverseString(char[] s) {
        int left = 0, right = s.length - 1;
        while (left < right) {
            char temp = s[left];
            s[left] = s[right];
            s[right] = temp;
            left++;
            right--;
        }
    }
}
```

23. Reverse Words in a String III

LeetCode #557 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two Pointer per Word (Split & Reverse)

Explanation

The words themselves must be reversed but their order in the sentence stays the same. Split the string on spaces to identify each word's boundaries, then apply the two-pointer reverse technique to each word individually, and finally join the reversed words back together with spaces.

Pseudocode

```
function reverseWords(s):
    words = split(s, " ")
    for i = 0 to length(words) - 1:
        words[i] = reverse(words[i])    // two-pointer reverse
    return join(words, " ")
```

Full Code (Java)

```
class Solution {
    public String reverseWords(String s) {
        String[] words = s.split(" ");
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < words.length; i++) {
            result.append(new StringBuilder(words[i]).reverse());
            if (i != words.length - 1) result.append(" ");
        }
        return result.toString();
    }
}
```

24. Length of Last Word

LeetCode #58 | Difficulty: **Easy**

Theorem / Technique Used

Reverse Linear Scan

Explanation

Scan the string from the end backward. First, skip any trailing spaces to find where the last word actually ends. Then continue scanning backward, counting characters, until another space or the start of the string is reached — that count is the length of the last word.

Pseudocode

```
function lengthOfLastWord(s):
    i = length(s) - 1
    while i >= 0 and s[i] == ' ':
        i = i - 1
    length = 0
    while i >= 0 and s[i] != ' ':
        length = length + 1
        i = i - 1
    return length
```

Full Code (Java)

```
class Solution {
    public int lengthOfLastWord(String s) {
        int i = s.length() - 1;
        while (i >= 0 && s.charAt(i) == ' ') i--;
        int length = 0;
        while (i >= 0 && s.charAt(i) != ' ') {
            length++;
            i--;
        }
        return length;
    }
}
```

25. Longest Common Prefix

LeetCode #14 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Horizontal Scanning (Prefix Shrinking)

Explanation

Take the first string in the array as an initial guess for the prefix. Compare it against every other string; whenever a mismatch is found, shorten the guessed prefix (chop off its last character) until it matches the start of that string, or becomes empty. Continue with the remaining strings.

Pseudocode

```
function longestCommonPrefix(strs):
    if strs is empty:
        return ""
    prefix = strs[0]
    for i = 1 to length(strs) - 1:
        while strs[i] does not start with prefix:
            prefix = prefix with last character removed
            if prefix is empty:
                return ""
    return prefix
```

Full Code (Java)

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        if (strs.length == 0) return "";
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (!strs[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
}
```

26. Roman to Integer

LeetCode #13 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Greedy Left-to-Right Scan with Lookahead

Explanation

Map each Roman numeral character to its value. Walk through the string left to right. Normally add the current symbol's value to the total. However, if the current symbol's value is smaller than the value of the symbol right after it, that means it is being used in a subtractive combination (like IV or IX), so subtract it instead of adding it.

Pseudocode

```
function romanToInt(s):
    values = {I:1, V:5, X:10, L:50, C:100, D:500, M:1000}
    total = 0
    for i = 0 to length(s) - 1:
        current = values[s[i]]
        if i + 1 < length(s) and current < values[s[i+1]]:
            total = total - current
        else:
            total = total + current
    return total
```

Full Code (Java)

```
class Solution {
    public int romanToInt(String s) {
        Map<Character, Integer> values = new HashMap<>();
        values.put('I', 1); values.put('V', 5); values.put('X', 10);
        values.put('L', 50); values.put('C', 100);
        values.put('D', 500); values.put('M', 1000);

        int total = 0;
        for (int i = 0; i < s.length(); i++) {
            int current = values.get(s.charAt(i));
            if (i + 1 < s.length() && current < values.get(s.charAt(i + 1))) {
                total -= current;
            } else {
                total += current;
            }
        }
        return total;
    }
}
```

27. Valid Palindrome

LeetCode #125 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two Pointer

Explanation

Use two pointers starting from both ends of the string. Skip over any character that is not a letter or digit. Compare the two valid characters case-insensitively; if they differ, the string is not a palindrome. Move both pointers inward and repeat until they meet.

Pseudocode

```
function isPalindrome(s):
    left = 0
    right = length(s) - 1
    while left < right:
        while left < right and not isAlphanumeric(s[left]):
            left = left + 1
        while left < right and not isAlphanumeric(s[right]):
            right = right - 1
        if lower(s[left]) != lower(s[right]):
            return false
        left = left + 1
        right = right - 1
    return true
```

Full Code (Java)

```
class Solution {
    public boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
            if (Character.toLowerCase(s.charAt(left)) != Character.toLowerCase(s.charAt(right))) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
}
```

28. First Unique Character in a String

LeetCode #387 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Frequency Counting (Two-Pass Scan)

Explanation

First pass: count how many times each character appears in the string (using a frequency array/map).

Second pass: scan the string again in order and return the index of the first character whose count is exactly

1. If none is found, return -1.

Pseudocode

```
function firstUniqChar(s):
    count = frequency map of each character in s
    for i = 0 to length(s) - 1:
        if count[s[i]] == 1:
            return i
    return -1
```

Full Code (Java)

```
class Solution {
    public int firstUniqChar(String s) {
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++;
        for (int i = 0; i < s.length(); i++) {
            if (count[s.charAt(i) - 'a'] == 1) return i;
        }
        return -1;
    }
}
```

29. Reverse Vowels of a String

LeetCode #345 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two Pointer

Explanation

Use two pointers from both ends of the string. Move the left pointer forward until it lands on a vowel, and move the right pointer backward until it lands on a vowel. Swap the two vowels, then move both pointers inward and repeat until they meet.

Pseudocode

```
function reverseVowels(s):
    vowels = {a,e,i,o,u,A,E,I,O,U}
    left = 0
    right = length(s) - 1
    while left < right:
        while left < right and s[left] not in vowels:
            left = left + 1
        while left < right and s[right] not in vowels:
            right = right - 1
        swap(s[left], s[right])
        left = left + 1
        right = right - 1
    return s
```

Full Code (Java)

```
class Solution {
    public String reverseVowels(String s) {
        Set<Character> vowels = new HashSet<>(Arrays.asList('a','e','i','o','u','A','E','I','O','U'));
        char[] arr = s.toCharArray();
        int left = 0, right = arr.length - 1;
        while (left < right) {
            while (left < right && !vowels.contains(arr[left])) left++;
            while (left < right && !vowels.contains(arr[right])) right--;
            char temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
        return new String(arr);
    }
}
```

30. Add Binary

LeetCode #67 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two Pointer with Carry (Simulated Addition)

Explanation

Simulate binary addition by hand, starting from the rightmost digits of both strings and moving left, carrying over any overflow. Use two pointers (one per string) that may run out at different times, plus a carry variable, and keep going until both pointers are exhausted and no carry remains.

Pseudocode

```
function addBinary(a, b):
    i = length(a) - 1
    j = length(b) - 1
    carry = 0
    result = ""
    while i >= 0 or j >= 0 or carry > 0:
        sum = carry
        if i >= 0: sum = sum + (a[i] - '0'); i = i - 1
        if j >= 0: sum = sum + (b[j] - '0'); j = j - 1
        result = (sum % 2) + result    // prepend digit
        carry = sum / 2
    return result
```

Full Code (Java)

```
class Solution {
    public String addBinary(String a, String b) {
        int i = a.length() - 1;
        int j = b.length() - 1;
        int carry = 0;
        StringBuilder result = new StringBuilder();
        while (i >= 0 || j >= 0 || carry > 0) {
            int sum = carry;
            if (i >= 0) sum += a.charAt(i--) - '0';
            if (j >= 0) sum += b.charAt(j--) - '0';
            result.append(sum % 2);
            carry = sum / 2;
        }
        return result.reverse().toString();
    }
}
```

Level 4 (Searching)

31. Binary Search

LeetCode #704 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Binary Search Theorem (Sorted Array Halving)

Explanation

Because the array is sorted, repeatedly compare the target with the middle element of the current search range. If the target equals the middle element, it's found. If the target is smaller, discard the right half by moving the 'high' pointer just before the middle; if larger, discard the left half by moving 'low' just after the middle. Repeat until the range is empty.

Pseudocode

```
function binarySearch(nums, target):
    low = 0
    high = length(nums) - 1
    while low <= high:
        mid = (low + high) / 2
        if nums[mid] == target:
            return mid
        else if nums[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Full Code (Java)

```
class Solution {
    public int search(int[] nums, int target) {
        int low = 0, high = nums.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
}
```

32. Search Insert Position

LeetCode #35 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Binary Search Theorem (Insertion Point Variant)

Explanation

This is standard binary search, but instead of returning -1 on failure, return the position where target would need to be inserted to keep the array sorted. That position is exactly where the 'low' pointer ends up once the loop finishes.

Pseudocode

```
function searchInsert(nums, target):
    low = 0
    high = length(nums) - 1
    while low <= high:
        mid = (low + high) / 2
        if nums[mid] == target:
            return mid
        else if nums[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return low // insertion point
```

Full Code (Java)

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int low = 0, high = nums.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return low;
    }
}
```

33. Guess Number Higher or Lower

LeetCode #374 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Binary Search Theorem (API-Guided)

Explanation

Use binary search over the range $[1, n]$. At each guess, call the provided `guess(num)` API, which returns 0 if correct, -1 if the pick is too high, or 1 if the pick is too low. Narrow the search range based on that feedback exactly like standard binary search.

Pseudocode

```
function guessNumber(n):
    low = 1
    high = n
    while low <= high:
        mid = (low + high) / 2
        result = guess(mid)
        if result == 0:
            return mid
        else if result < 0:      // mid is too high
            high = mid - 1
        else:                  // mid is too low
            low = mid + 1
```

Full Code (Java)

```
/* The guess API is defined in the parent class GuessGame.
   int guess(int num); */

public class Solution extends GuessGame {
    public int guessNumber(int n) {
        int low = 1, high = n;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            int result = guess(mid);
            if (result == 0) return mid;
            else if (result < 0) high = mid - 1;
            else low = mid + 1;
        }
        return -1;
    }
}
```

34. Square Root(x)

LeetCode #69 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Binary Search Theorem (Search on Answer Space)

Explanation

Binary search over the range of possible answers, from 0 to x. For each candidate mid, check if mid*mid is less than or equal to x. Keep track of the largest mid that satisfies this condition, since the answer is the floor of the true square root.

Pseudocode

```
function mySqrt(x):
    low = 0
    high = x
    answer = 0
    while low <= high:
        mid = (low + high) / 2
        if mid * mid <= x:
            answer = mid
            low = mid + 1
        else:
            high = mid - 1
    return answer
```

Full Code (Java)

```
class Solution {
    public int mySqrt(int x) {
        long low = 0, high = x;
        long answer = 0;
        while (low <= high) {
            long mid = low + (high - low) / 2;
            if (mid * mid <= x) {
                answer = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return (int) answer;
    }
}
```

35. First Bad Version

LeetCode #278 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Binary Search Theorem (First True Boundary)

Explanation

Binary search over versions 1..n. Use the provided `isBadVersion(version)` API as the comparison. If a version is bad, the first bad version is at that index or earlier, so search the left half (including mid); if it is good, search strictly to the right. The loop converges on the first bad version.

Pseudocode

```
function firstBadVersion(n):
    low = 1
    high = n
    while low < high:
        mid = low + (high - low) / 2
        if isBadVersion(mid):
            high = mid
        else:
            low = mid + 1
    return low
```

Full Code (Java)

```
/* The isBadVersion API is defined in the parent class VersionControl.
   boolean isBadVersion(int version); */

public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int low = 1, high = n;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (isBadVersion(mid)) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
        return low;
    }
}
```

Level 5 (Recursion)

36. Fibonacci Number

LeetCode #509 | Difficulty: **Easy**

Theorem / Technique Used

Recursion (Fibonacci Recurrence)

Explanation

The Fibonacci sequence is defined recursively: $F(0)=0$, $F(1)=1$, and $F(n) = F(n-1) + F(n-2)$ for $n>1$. Translate this definition directly into a recursive function with two base cases and one recursive step that calls itself twice.

Pseudocode

```
function fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fib(n - 1) + fib(n - 2)
```

Full Code (Java)

```
class Solution {
    public int fib(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        return fib(n - 1) + fib(n - 2);
    }
}
```

37. Power of Two

LeetCode #231 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Recursion (Repeated Halving)

Explanation

A number is a power of two if it can be repeatedly divided by 2 until it reaches exactly 1, with no remainder at any step. Handle non-positive numbers as false immediately, then recurse: if n is 1, it's a power of two; if n is odd (and not 1) it can't be; otherwise recurse on $n/2$.

Pseudocode

```
function isPowerOfTwo(n):
    if n <= 0:
        return false
    if n == 1:
        return true
    if n % 2 != 0:
        return false
    return isPowerOfTwo(n / 2)
```

Full Code (Java)

```
class Solution {
    public boolean isPowerOfTwo(int n) {
        if (n <= 0) return false;
        if (n == 1) return true;
        if (n % 2 != 0) return false;
        return isPowerOfTwo(n / 2);
    }
}
```

38. Climbing Stairs

LeetCode #70 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Recursion (Fibonacci Recurrence)

Explanation

To reach step n , the last move was either a single step from $n-1$, or a double step from $n-2$. So the number of ways to reach step n is the sum of the ways to reach $n-1$ and $n-2$ — the same recurrence as Fibonacci. Base cases: 1 way to reach step 0 or step 1.

Pseudocode

```
function climbStairs(n):
    if n <= 1:
        return 1
    return climbStairs(n - 1) + climbStairs(n - 2)
// Note: plain recursion is exponential; adding memoization
// (storing already-computed results) makes it efficient.
```

Full Code (Java)

```
class Solution {
    public int climbStairs(int n) {
        if (n <= 1) return 1;
        return climbStairs(n - 1) + climbStairs(n - 2);
        // For large n, add memoization (HashMap/array cache) to avoid
        // exponential time complexity.
    }
}
```

39. Pow(x,n)

LeetCode #50 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Fast Exponentiation (Divide and Conquer)

Explanation

Use fast exponentiation (divide and conquer) instead of multiplying x by itself n times. To compute x^n , recursively compute $\text{half} = x^{(n/2)}$, then combine: if n is even, result is $\text{half} * \text{half}$; if n is odd, multiply in one extra x . Handle negative exponents by computing $1 / x^{(-n)}$.

Pseudocode

```
function myPow(x, n):
    if n < 0:
        return 1 / myPow(x, -n)
    if n == 0:
        return 1
    half = myPow(x, n / 2)
    if n % 2 == 0:
        return half * half
    else:
        return half * half * x
```

Full Code (Java)

```
class Solution {
    public double myPow(double x, int n) {
        long N = n;
        if (N < 0) {
            x = 1 / x;
            N = -N;
        }
        return fastPow(x, N);
    }

    private double fastPow(double x, long n) {
        if (n == 0) return 1.0;
        double half = fastPow(x, n / 2);
        if (n % 2 == 0) return half * half;
        else return half * half * x;
    }
}
```

40. Recursive Binary Search

Not separately numbered on LeetCode | Difficulty: **Easy-Medium**

Theorem / Technique Used

Binary Search Theorem (Recursive Form)

Explanation

Same idea as the iterative binary search, but expressed with recursive calls instead of a while loop. Each call narrows the search range in half and calls itself on the remaining half, stopping when the range is empty (base case) or the target is found.

Pseudocode

```
function binarySearchRecursive(nums, target, low, high):
    if low > high:
        return -1
    mid = (low + high) / 2
    if nums[mid] == target:
        return mid
    else if nums[mid] < target:
        return binarySearchRecursive(nums, target, mid + 1, high)
    else:
        return binarySearchRecursive(nums, target, low, mid - 1)
```

Full Code (Java)

```
class Solution {
    public int binarySearchRecursive(int[] nums, int target, int low, int high) {
        if (low > high) return -1;
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) return binarySearchRecursive(nums, target, mid + 1, high);
        else return binarySearchRecursive(nums, target, low, mid - 1);
    }
}
```

Level 6 (Intermediate)

41. Pascal's Triangle

LeetCode #118 | Difficulty: **Easy**

Theorem / Technique Used

Pascal's Rule (Binomial Coefficient Recurrence)

Explanation

Build the triangle row by row. The first row is always [1]. Each subsequent row starts and ends with 1; every interior value is the sum of the two values above it (from the previous row). Store each completed row before building the next.

Pseudocode

```
function generate(numRows):
    triangle = []
    for i = 0 to numRows - 1:
        row = new array of size i+1, filled with 1
        for j = 1 to i - 1:
            row[j] = triangle[i-1][j-1] + triangle[i-1][j]
        triangle.append(row)
    return triangle
```

Full Code (Java)

```
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> triangle = new ArrayList<>();
        for (int i = 0; i < numRows; i++) {
            List<Integer> row = new ArrayList<>();
            for (int j = 0; j <= i; j++) row.add(1);
            for (int j = 1; j < i; j++) {
                row.set(j, triangle.get(i - 1).get(j - 1) + triangle.get(i - 1).get(j));
            }
            triangle.add(row);
        }
        return triangle;
    }
}
```

42. Pascal's Triangle II

LeetCode #119 | Difficulty: **Easy**

Theorem / Technique Used

Pascal's Rule (In-Place Right-to-Left Update)

Explanation

Only the row at the given index is needed, not the whole triangle. Build rows iteratively from the top, but only keep the current row in memory (or overwrite it in place from right to left so earlier values in the same row aren't lost before they're used).

Pseudocode

```
function getRow(rowIndex):
    row = array of size rowIndex+1, filled with 1
    for i = 1 to rowIndex:
        for j = i - 1 down to 1:
            row[j] = row[j] + row[j-1]
    return row
```

Full Code (Java)

```
class Solution {
    public List<Integer> getRow(int rowIndex) {
        List<Integer> row = new ArrayList<>();
        for (int i = 0; i <= rowIndex; i++) row.add(1);
        for (int i = 1; i <= rowIndex; i++) {
            for (int j = i - 1; j >= 1; j--) {
                row.set(j, row.get(j) + row.get(j - 1));
            }
        }
        return row;
    }
}
```

43. Happy Number

LeetCode #202 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Cycle Detection (Set-Based)

Explanation

Repeatedly replace the number with the sum of the squares of its digits. If this process reaches 1, the number is happy. If it enters a cycle instead of reaching 1, it is not happy. Use a set to record every number seen so far; if a number repeats, a cycle has been detected and the answer is false.

Pseudocode

```
function isHappy(n):
    seen = empty set
    while n != 1 and n not in seen:
        seen.add(n)
        n = sumOfSquaredDigits(n)
    return n == 1

function sumOfSquaredDigits(n):
    total = 0
    while n > 0:
        digit = n % 10
        total = total + digit * digit
        n = n / 10
    return total
```

Full Code (Java)

```
class Solution {
    public boolean isHappy(int n) {
        Set<Integer> seen = new HashSet<>();
        while (n != 1 && !seen.contains(n)) {
            seen.add(n);
            n = sumOfSquaredDigits(n);
        }
        return n == 1;
    }

    private int sumOfSquaredDigits(int n) {
        int total = 0;
        while (n > 0) {
            int digit = n % 10;
            total += digit * digit;
            n /= 10;
        }
        return total;
    }
}
```

44. Missing Number

LeetCode #268 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Gauss's Summation Formula

Explanation

The array contains n distinct numbers from the range $0..n$, with exactly one missing. Compute what the sum of $0..n$ should be using the formula $n*(n+1)/2$, then subtract the actual sum of the array's elements. The difference is the missing number.

Pseudocode

```
function missingNumber(nums):
    n = length(nums)
    expectedSum = n * (n + 1) / 2
    actualSum = sum(nums)
    return expectedSum - actualSum
```

Full Code (Java)

```
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;
        for (int num : nums) actualSum += num;
        return expectedSum - actualSum;
    }
}
```

45. Move Zeroes

LeetCode #283 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two Pointer (Slow/Fast)

Explanation

Use a slow pointer that marks where the next non-zero value should go. Scan through the array with a fast pointer; whenever a non-zero value is found, place it at the slow pointer's position and advance the slow pointer. After the scan, fill every remaining position from the slow pointer to the end with zeroes.

Pseudocode

```
function moveZeroes(nums):
    slow = 0
    for fast = 0 to length(nums) - 1:
        if nums[fast] != 0:
            nums[slow] = nums[fast]
            slow = slow + 1
    for i = slow to length(nums) - 1:
        nums[i] = 0
```

Full Code (Java)

```
class Solution {
    public void moveZeroes(int[] nums) {
        int slow = 0;
        for (int fast = 0; fast < nums.length; fast++) {
            if (nums[fast] != 0) {
                nums[slow] = nums[fast];
                slow++;
            }
        }
        for (int i = slow; i < nums.length; i++) {
            nums[i] = 0;
        }
    }
}
```

46. Rotate Array

LeetCode #189 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Reversal Algorithm (Juggling / Block Swap Alternative)

Explanation

Rotating the array right by k positions can be done using the 'reverse trick': reverse the entire array, then reverse the first k elements, then reverse the remaining $n-k$ elements. This achieves the rotation in place without needing extra array space (aside from $O(1)$ helper variables).

Pseudocode

```
function rotate(nums, k):
    n = length(nums)
    k = k % n
    reverse(nums, 0, n - 1)
    reverse(nums, 0, k - 1)
    reverse(nums, k, n - 1)

function reverse(nums, left, right):
    while left < right:
        swap(nums[left], nums[right])
        left = left + 1
        right = right - 1
```

Full Code (Java)

```
class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k %= n;
        reverse(nums, 0, n - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }

    private void reverse(int[] nums, int left, int right) {
        while (left < right) {
            int temp = nums[left];
            nums[left] = nums[right];
            nums[right] = temp;
            left++;
            right--;
        }
    }
}
```

47. Third Maximum Number

LeetCode #414 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Constant-Space Top-K Tracking

Explanation

Keep track of the top three distinct maximum values seen so far using three variables (first, second, third), initialized to negative infinity. Scan the array once; for each number, update the three trackers so they always hold the largest, second largest, and third largest distinct values. If the third tracker was never updated, return the first (maximum) instead.

Pseudocode

```
function thirdMax(nums):
    first = second = third = -infinity
    for each num in nums:
        if num == first or num == second or num == third:
            continue
        if num > first:
            third = second; second = first; first = num
        else if num > second:
            third = second; second = num
        else if num > third:
            third = num
    if third == -infinity:
        return first
    return third
```

Full Code (Java)

```
class Solution {
    public int thirdMax(int[] nums) {
        Long first = null, second = null, third = null;
        for (int num : nums) {
            long n = num;
            if ((first != null && n == first) || (second != null && n == second) ||
                (third != null && n == third)) continue;
            if (first == null || n > first) {
                third = second; second = first; first = n;
            } else if (second == null || n > second) {
                third = second; second = n;
            } else if (third == null || n > third) {
                third = n;
            }
        }
        return third == null ? first.intValue() : third.intValue();
    }
}
```

48. Find All Numbers Disappeared in Array

LeetCode #448 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Index-as-Hash-Key (In-Place Marking)

Explanation

The array holds values from 1..n, with some appearing more than once and others missing entirely. Use each value as an index: for every number x in the array, mark the value at index $|x|-1$ as negative (without changing which numbers are read next, using absolute value to read). After processing, any index whose value is still positive corresponds to a missing number (index+1).

Pseudocode

```
function findDisappearedNumbers(nums):
    for each num in nums:
        index = abs(num) - 1
        if nums[index] > 0:
            nums[index] = -nums[index]
    result = []
    for i = 0 to length(nums) - 1:
        if nums[i] > 0:
            result.append(i + 1)
    return result
```

Full Code (Java)

```
class Solution {
    public List<Integer> findDisappearedNumbers(int[] nums) {
        for (int num : nums) {
            int index = Math.abs(num) - 1;
            if (nums[index] > 0) nums[index] = -nums[index];
        }
        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] > 0) result.add(i + 1);
        }
        return result;
    }
}
```

49. Valid Mountain Array

LeetCode #941 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Two-Direction Linear Scan

Explanation

A valid mountain strictly increases to a single peak and then strictly decreases, with the peak not at either end. Walk up from the left as long as values are strictly increasing, then walk down from the right as long as values are strictly increasing (from the right side). Both walks must meet at the same peak index, which must not be the first or last element.

Pseudocode

```
function validMountainArray(arr):
    n = length(arr)
    i = 0
    while i + 1 < n and arr[i] < arr[i+1]:
        i = i + 1
    if i == 0 or i == n - 1:
        return false
    while i + 1 < n and arr[i] > arr[i+1]:
        i = i + 1
    return i == n - 1
```

Full Code (Java)

```
class Solution {
    public boolean validMountainArray(int[] arr) {
        int n = arr.length;
        int i = 0;
        while (i + 1 < n && arr[i] < arr[i + 1]) i++;
        if (i == 0 || i == n - 1) return false;
        while (i + 1 < n && arr[i] > arr[i + 1]) i++;
        return i == n - 1;
    }
}
```

50. Monotonic Array

LeetCode #896 | Difficulty: **Easy-Medium**

Theorem / Technique Used

Single-Pass Dual-Flag Scan

Explanation

An array is monotonic if it is entirely non-decreasing or entirely non-increasing. Make a single pass while tracking two flags: whether any adjacent pair breaks the non-decreasing rule, and whether any adjacent pair breaks the non-increasing rule. The array is monotonic if at least one of the two flags never gets triggered.

Pseudocode

```
function isMonotonic(nums):
    increasing = true
    decreasing = true
    for i = 1 to length(nums) - 1:
        if nums[i] < nums[i-1]:
            increasing = false
        if nums[i] > nums[i-1]:
            decreasing = false
    return increasing or decreasing
```

Full Code (Java)

```
class Solution {
    public boolean isMonotonic(int[] nums) {
        boolean increasing = true, decreasing = true;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] < nums[i - 1]) increasing = false;
            if (nums[i] > nums[i - 1]) decreasing = false;
        }
        return increasing || decreasing;
    }
}
```

Hardest Questions Possible with Only Module 1 Knowledge

51. Longest Palindromic Substring

LeetCode #5 | Difficulty: **Hard**

Theorem / Technique Used

Expand Around Center

Explanation

Module-1-only approach: 'expand around center'. Every palindrome has a center (a single character for odd-length palindromes, or a gap between two characters for even-length ones). For each possible center in the string, expand outward in both directions while the characters match, and track the longest palindrome found across all centers. This only needs loops and comparisons — no advanced data structures.

Pseudocode

```
function longestPalindrome(s):
    start = 0
    maxLength = 1
    for center = 0 to length(s) - 1:
        len1 = expandAroundCenter(s, center, center)      // odd length
        len2 = expandAroundCenter(s, center, center + 1)  // even length
        len = max(len1, len2)
        if len > maxLength:
            maxLength = len
            start = center - (len - 1) / 2
    return substring(s, start, start + maxLength)

function expandAroundCenter(s, left, right):
    while left >= 0 and right < length(s) and s[left] == s[right]:
        left = left - 1
        right = right + 1
    return right - left - 1
```

Full Code (Java)

```
class Solution {
    public String longestPalindrome(String s) {
        if (s.length() < 1) return "";
        int start = 0, maxLength = 1;
        for (int center = 0; center < s.length(); center++) {
            int len1 = expandAroundCenter(s, center, center);
            int len2 = expandAroundCenter(s, center, center + 1);
            int len = Math.max(len1, len2);
            if (len > maxLength) {
                maxLength = len;
                start = center - (len - 1) / 2;
            }
        }
        return s.substring(start, start + maxLength);
    }

    private int expandAroundCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1;
    }
}
```

52. Zigzag Conversion

LeetCode #6 | Difficulty: **Hard**

Theorem / Technique Used

Simulation (Row Bouncing Pattern)

Explanation

Simulate the zigzag writing process directly using an array of string 'rows'. Walk through the input string character by character, appending each character to the row matching the current pointer position. Move the row pointer down until hitting the last row, then reverse direction and move up until hitting the first row, repeating this bounce for the whole string. Finally, concatenate all rows.

Pseudocode

```
function convert(s, numRows):
    if numRows == 1:
        return s
    rows = array of numRows empty strings
    currentRow = 0
    goingDown = false
    for each ch in s:
        rows[currentRow] = rows[currentRow] + ch
        if currentRow == 0 or currentRow == numRows - 1:
            goingDown = not goingDown
        currentRow = currentRow + (1 if goingDown else -1)
    return concatenate(rows)
```

Full Code (Java)

```
class Solution {
    public String convert(String s, int numRows) {
        if (numRows == 1) return s;
        StringBuilder[] rows = new StringBuilder[numRows];
        for (int i = 0; i < numRows; i++) rows[i] = new StringBuilder();

        int currentRow = 0;
        boolean goingDown = false;
        for (char ch : s.toCharArray()) {
            rows[currentRow].append(ch);
            if (currentRow == 0 || currentRow == numRows - 1) goingDown = !goingDown;
            currentRow += goingDown ? 1 : -1;
        }
        StringBuilder result = new StringBuilder();
        for (StringBuilder row : rows) result.append(row);
        return result.toString();
    }
}
```

53. String to Integer (atoi)

LeetCode #8 | Difficulty: **Hard**

Theorem / Technique Used

Finite State Simulation (Manual Parsing)

Explanation

Carefully simulate the parsing rules step by step using only loops and if-else: (1) skip any leading whitespace; (2) read an optional '+' or '-' sign; (3) read digits one at a time, building the number with $\text{num} = \text{num} * 10 + \text{digit}$, stopping at the first non-digit character; (4) clamp the final result to the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$.

Pseudocode

```
function myAtoi(s):
    i = 0
    n = length(s)
    while i < n and s[i] == ' ':
        i = i + 1
    sign = 1
    if i < n and (s[i] == '+' or s[i] == '-'):
        if s[i] == '-':
            sign = -1
        i = i + 1
    result = 0
    while i < n and isDigit(s[i]):
        result = result * 10 + (s[i] - '0')
        if result * sign > 2^31 - 1:
            return 2^31 - 1
        if result * sign < -2^31:
            return -2^31
        i = i + 1
    return result * sign
```

Full Code (Java)

```
class Solution {
    public int myAtoi(String s) {
        int i = 0, n = s.length();
        while (i < n && s.charAt(i) == ' ') i++;
        int sign = 1;
        if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
            if (s.charAt(i) == '-') sign = -1;
            i++;
        }
        long result = 0;
        while (i < n && Character.isDigit(s.charAt(i))) {
            result = result * 10 + (s.charAt(i) - '0');
            if (result * sign > Integer.MAX_VALUE) return Integer.MAX_VALUE;
            if (result * sign < Integer.MIN_VALUE) return Integer.MIN_VALUE;
            i++;
        }
        return (int) (result * sign);
    }
}
```

54. Container With Most Water

LeetCode #11 | Difficulty: **Hard**

Theorem / Technique Used

Two Pointer (Greedy Shrinking)

Explanation

Use two pointers starting at the two ends of the height array. At each step, compute the area between the two pointers (width times the shorter of the two heights) and track the maximum. Since the container is always limited by the shorter side, move the pointer at the shorter height inward — moving the taller one could only shrink the width without any chance of increasing the limiting height.

Pseudocode

```
function maxArea(height):
    left = 0
    right = length(height) - 1
    best = 0
    while left < right:
        width = right - left
        area = width * min(height[left], height[right])
        best = max(best, area)
        if height[left] < height[right]:
            left = left + 1
        else:
            right = right - 1
    return best
```

Full Code (Java)

```
class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1;
        int best = 0;
        while (left < right) {
            int width = right - left;
            int area = width * Math.min(height[left], height[right]);
            best = Math.max(best, area);
            if (height[left] < height[right]) left++;
            else right--;
        }
        return best;
    }
}
```

55. 3Sum

LeetCode #15 | Difficulty: **Hard**

Theorem / Technique Used

Sorting + Two Pointer

Explanation

Sort the array first. Fix one element with an outer loop, then use two pointers (left just after the fixed element, right at the end) to find pairs that sum with the fixed element to zero, moving the pointers inward based on whether the running sum is too high or too low. Skip over duplicate values at every level to avoid duplicate triplets in the result.

Pseudocode

```
function threeSum(nums):
    sort(nums)
    result = []
    for i = 0 to length(nums) - 1:
        if i > 0 and nums[i] == nums[i-1]:
            continue // skip duplicate fixed element
        left = i + 1
        right = length(nums) - 1
        while left < right:
            sum = nums[i] + nums[left] + nums[right]
            if sum == 0:
                result.append([nums[i], nums[left], nums[right]])
                while left < right and nums[left] == nums[left+1]: left = left + 1
                while left < right and nums[right] == nums[right-1]: right = right - 1
                left = left + 1
                right = right - 1
            else if sum < 0:
                left = left + 1
            else:
                right = right - 1
    return result
```

Full Code (Java)

```
import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int left = i + 1, right = nums.length - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                    while (left < right && nums[left] == nums[left + 1]) left++;
                    while (left < right && nums[right] == nums[right - 1]) right--;
                    left++;
                    right--;
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return result;
    }
}
```

56. Generate Parentheses

LeetCode #22 | Difficulty: **Hard**

Theorem / Technique Used

Recursive Backtracking

Explanation

Use recursive backtracking to build strings character by character. Track how many '(' and ')' have been used so far. At each step, an open paren can be added if fewer than n have been used; a close paren can be added only if fewer close parens have been used than open ones (this guarantees validity at every point). Once the string reaches length $2n$, it's added to the result.

Pseudocode

```
function generateParenthesis(n):
    result = []
    backtrack("", 0, 0, n, result)
    return result

function backtrack(current, openCount, closeCount, n, result):
    if length(current) == 2 * n:
        result.append(current)
        return
    if openCount < n:
        backtrack(current + "(", openCount + 1, closeCount, n, result)
    if closeCount < openCount:
        backtrack(current + ")", openCount, closeCount + 1, n, result)
```

Full Code (Java)

```
class Solution {
    public List<String> generateParenthesis(int n) {
        List<String> result = new ArrayList<>();
        backtrack(new StringBuilder(), 0, 0, n, result);
        return result;
    }

    private void backtrack(StringBuilder current, int open, int close, int n, List<String> result) {
        if (current.length() == 2 * n) {
            result.add(current.toString());
            return;
        }
        if (open < n) {
            current.append('(');
            backtrack(current, open + 1, close, n, result);
            current.deleteCharAt(current.length() - 1);
        }
        if (close < open) {
            current.append(')');
            backtrack(current, open, close + 1, n, result);
            current.deleteCharAt(current.length() - 1);
        }
    }
}
```

57. Letter Combinations of a Phone Number

LeetCode #17 | Difficulty: **Hard**

Theorem / Technique Used

Recursive Backtracking

Explanation

Use recursive backtracking with a lookup table mapping each digit to its letters (e.g. 2 -> 'abc'). At each recursive step, pick the next digit's letters one at a time, append each letter to the in-progress combination, and recurse on the remaining digits. When all digits have been used, the completed combination is added to the result.

Pseudocode

```
function letterCombinations(digits):
    if digits is empty:
        return []
    letters = {2:"abc",3:"def",4:"ghi",5:"jkl",6:"mno",7:"pqrs",8:"tuv",9:"wxyz"}
    result = []
    backtrack("", 0, digits, letters, result)
    return result

function backtrack(current, index, digits, letters, result):
    if index == length(digits):
        result.append(current)
        return
    possibleLetters = letters[digits[index]]
    for each letter in possibleLetters:
        backtrack(current + letter, index + 1, digits, letters, result)
```

Full Code (Java)

```
class Solution {
    public List<String> letterCombinations(String digits) {
        List<String> result = new ArrayList<>();
        if (digits.isEmpty()) return result;
        Map<Character, String> letters = new HashMap<>();
        letters.put('2', "abc"); letters.put('3', "def"); letters.put('4', "ghi");
        letters.put('5', "jkl"); letters.put('6', "mno"); letters.put('7', "pqrs");
        letters.put('8', "tuv"); letters.put('9', "wxyz");
        backtrack(new StringBuilder(), 0, digits, letters, result);
        return result;
    }

    private void backtrack(StringBuilder current, int index, String digits,
                           Map<Character, String> letters, List<String> result) {
        if (index == digits.length()) {
            result.add(current.toString());
            return;
        }
        String possibleLetters = letters.get(digits.charAt(index));
        for (char letter : possibleLetters.toCharArray()) {
            current.append(letter);
            backtrack(current, index + 1, digits, letters, result);
            current.deleteCharAt(current.length() - 1);
        }
    }
}
```

58. Permutations

LeetCode #46 | Difficulty: **Hard**

Theorem / Technique Used

Recursive Backtracking

Explanation

Use recursive backtracking: maintain a 'current' partial permutation and a marker of which numbers have already been used. At each step, try adding every unused number to the current permutation, recurse deeper, and then undo the choice (backtrack) to try the next option. When the current permutation includes every number, save a copy to the result.

Pseudocode

```
function permute(nums):
    result = []
    used = array of length(nums) booleans, all false
    backtrack([], used, nums, result)
    return result

function backtrack(current, used, nums, result):
    if length(current) == length(nums):
        result.append(copy of current)
        return
    for i = 0 to length(nums) - 1:
        if used[i]:
            continue
        used[i] = true
        current.append(nums[i])
        backtrack(current, used, nums, result)
        current.removeLast()    // undo
        used[i] = false
```

Full Code (Java)

```
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        boolean[] used = new boolean[nums.length];
        backtrack(new ArrayList<>(), used, nums, result);
        return result;
    }

    private void backtrack(List<Integer> current, boolean[] used, int[] nums,
                           List<List<Integer>> result) {
        if (current.size() == nums.length) {
            result.add(new ArrayList<>(current));
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            current.add(nums[i]);
            backtrack(current, used, nums, result);
            current.remove(current.size() - 1);
            used[i] = false;
        }
    }
}
```

59. Combination Sum

LeetCode #39 | Difficulty: **Hard**

Theorem / Technique Used

Recursive Backtracking

Explanation

Use recursive backtracking: at each step, choose whether to include the current candidate again (since reuse is allowed) or move to the next candidate. Subtract the chosen value from the remaining target; if the remaining target hits exactly zero, save the current combination. If it goes below zero, stop that branch (prune). Always move the starting index forward when skipping a candidate so combinations are not generated in different orders.

Pseudocode

```
function combinationSum(candidates, target):
    result = []
    backtrack([], 0, target, candidates, result)
    return result

function backtrack(current, start, remaining, candidates, result):
    if remaining == 0:
        result.append(copy of current)
        return
    if remaining < 0:
        return
    for i = start to length(candidates) - 1:
        current.append(candidates[i])
        backtrack(current, i, remaining - candidates[i], candidates, result) // i, not i+1
    : reuse allowed
    current.removeLast()
```

Full Code (Java)

```
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(new ArrayList<>(), 0, target, candidates, result);
        return result;
    }

    private void backtrack(List<Integer> current, int start, int remaining,
                           int[] candidates, List<List<Integer>> result) {
        if (remaining == 0) {
            result.add(new ArrayList<>(current));
            return;
        }
        if (remaining < 0) return;
        for (int i = start; i < candidates.length; i++) {
            current.add(candidates[i]);
            backtrack(current, i, remaining - candidates[i], candidates, result);
            current.remove(current.size() - 1);
        }
    }
}
```

60. Sudoku Solver

LeetCode #37 | Difficulty: **Very Hard**

Theorem / Technique Used

Recursive Backtracking (Constraint Satisfaction)

Explanation

Use recursive backtracking over empty cells. For each empty cell, try placing digits 1-9; before placing, check that the digit does not already appear in the same row, column, or 3x3 box. If valid, place it and recurse to the next empty cell. If the recursion fails (no digit works anywhere further down), undo the placement (backtrack) and try the next digit. The board is solved once every cell is filled validly.

Pseudocode

```
function solveSudoku(board):
    return solve(board)

function solve(board):
    cell = findNextEmptyCell(board)
    if cell is null:
        return true    // no empty cells left -> solved
    row, col = cell
    for digit = 1 to 9:
        if isValidPlacement(board, row, col, digit):
            board[row][col] = digit
            if solve(board):
                return true
            board[row][col] = 0    // undo (backtrack)
    return false    // trigger backtracking on caller

function isValidPlacement(board, row, col, digit):
    check row, column, and 3x3 box for existing 'digit'
    return true only if digit is not already present in any of them
```

Full Code (Java)

```
class Solution {
    public void solveSudoku(char[][] board) {
        solve(board);
    }

    private boolean solve(char[][] board) {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == '.') {
                    for (char digit = '1'; digit <= '9'; digit++) {
                        if (isValidPlacement(board, row, col, digit)) {
                            board[row][col] = digit;
                            if (solve(board)) return true;
                            board[row][col] = '.';
                        }
                    }
                    return false;
                }
            }
        }
        return true;
    }

    private boolean isValidPlacement(char[][] board, int row, int col, char digit) {
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == digit) return false;
            if (board[i][col] == digit) return false;
            int boxRow = 3 * (row / 3) + i / 3;
            int boxCol = 3 * (col / 3) + i % 3;
            if (board[boxRow][boxCol] == digit) return false;
        }
        return true;
    }
}
```